

Graph-Temporal Intrusion Detection for IoT: A Heterogeneous Ensemble Approach

Iacovos Ioannou^{§†*}, Michael Georgiades[¶], Anastasia Ioannou[§], Konstantinos Katzis[§],
Yianna Danidou[§], Katerina Papanikolaou[§], Marios Raspopoulos[‡],

Christophoros Christophorou[‡] Michalis Savva^{*†}, and Vasos Vassiliou^{*†}

[§]Department of Computer Science and Engineering, European University Cyprus, Nicosia, Cyprus

^{*}Department of Computer Science, University of Cyprus, Nicosia, Cyprus

[†]CYENS - Centre of Excellence, Nicosia, Cyprus

[‡]UCLan Cyprus, Pyla, Cyprus

[¶]Department of Computer Science, Neapolis University Pafos, Pafos, Cyprus

Abstract—The proliferation of IoT devices creates a vast attack surface requiring intrusion detection systems (IDS) that balance high accuracy with the computational constraints of edge environments. This paper addresses the problem of achieving robust security within strict resource limits. We introduce a comprehensive benchmarking framework to jointly evaluate detection quality and computational efficiency. Our core contribution is the *Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE)*, a novel two-view ensemble. It strategically combines lightweight tree-based models with more powerful, but computationally expensive, deep sequence models only when necessary to optimize both detection performance and inference speed.

Using the BoT-IoT dataset, we apply multi-stage preprocessing and enrich features with graph-temporal summaries capturing inter-device patterns and temporal dynamics. Results show that while traditional tree-based methods (Random Forest, XGBoost) saturate at 0.99 accuracy, our GTA-HE is the only architecture to achieve 1.00 accuracy and F1 scores (two-decimal precision) due to the selected dataset’s relatively distinct feature values. The ensemble successfully corrects the false negatives produced by lighter models when faced with complex attack patterns. In contrast, standalone deep sequence models show poor minority-class recall while requiring orders of magnitude more computation time. The proposed GTA-HE achieves 1.00 accuracy with exceptional computational efficiency, achieving an inference time of only 5.90 ms and a throughput exceeding 6.2 million samples per second, indicating that our approach could be used in real-time-critical systems. Our work demonstrates that sophisticated, selective ensembles like GTA-HE can deliver superior security without compromising the operational efficiency required at the edge.

Index Terms—Internet of Things, intrusion detection systems, computational efficiency, graph-temporal features, heterogeneous ensemble, BoT-IoT dataset, edge computing

I. INTRODUCTION

The unprecedented growth of Internet of Things (IoT) deployments has fundamentally transformed modern computing, with over 75 billion connected devices projected by 2025 [1], [2]. This explosive expansion has created an attack surface of monumental proportions, characterized by heterogeneous device types, diverse communication protocols, and sophisticated attack vectors ranging from distributed denial-of-service (DDoS) and data exfiltration to complete system compromise and botnet recruitment. Unlike traditional enterprise networks, IoT environments present unique security challenges: resource-constrained hardware, limited computational capabilities, and

devices often deployed without adequate security considerations [3]–[5]. Intrusion detection systems (IDS) serve as a critical defense layer in IoT ecosystems, monitoring network traffic to identify malicious patterns and security breaches [6]–[8]. However, deploying effective IDS on IoT devices demands careful consideration of computational complexity, memory footprint, and latency constraints—requirements often orthogonal to detection accuracy objectives.

Contemporary IoT intrusion detection research predominantly focuses on maximizing detection accuracy through increasingly complex machine learning and deep learning architectures, while paying insufficient attention to computational overhead and latency constraints [9], [10]. Many state-of-the-art approaches employ deep learning models that, while achieving impressive accuracy on conventional datasets, incur prohibitive computational costs during inference on resource-limited edge devices [11]–[13]. Conversely, simpler algorithms designed for efficiency often sacrifice detection performance, hitting a performance ceiling on sophisticated attacks and minority-class patterns, thus creating significant security gaps (i.e., elevated false negative rates for novel attack variants, inadequate detection of zero-day exploits, and poor generalization to evolving threat landscapes) [14]–[17].

This paper addresses this fundamental gap by proposing a systematic benchmarking framework that treats inference-time efficiency (i.e., latency and throughput) as a first-class design objective alongside detection accuracy, achieving both rapid threat identification and robust security without compromising either objective. We introduce a comprehensive evaluation methodology that jointly assesses security effectiveness and resource consumption, enabling informed trade-off decisions for specific IoT deployment contexts. Beyond the benchmarking framework, we introduce the *Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE)* that strategically combines lightweight tree-based classifiers with computationally expensive deep sequence models. This ensemble employs an intelligent gating mechanism that routes straightforward samples through efficient tree-based models while reserving deep learning analysis for ambiguous cases, achieving superior overall performance without the constant computational burden of deep architectures.

Several critical observations motivate this research. First,

the dramatic increase in IoT-specific cyberattacks, including the Mirai botnet that compromised hundreds of thousands of devices, demonstrates the urgent need for effective, deployable security solutions. Second, the resource asymmetry between attackers and defenders in IoT environments creates fundamental challenges. Attackers leverage cloud-scale resources to develop attacks, while defense mechanisms must operate within severe constraints of edge devices. Third, the diversity of IoT deployment scenarios from industrial control to smart homes to healthcare monitoring requires security solutions tailored to specific operational constraints. A one-size-fits-all approach fails to address the nuanced requirements of different IoT applications. Fourth, current research evaluation practices often prioritize detection accuracy metrics while ignoring practical deployment considerations. Models achieving 99% accuracy in research settings may be entirely impractical for real-world IoT deployment due to computational demands, while still missing the critical 1% of advanced attacks. The main contributions of this work are:

- 1) **Comprehensive Benchmarking Framework:** A systematic methodology that jointly evaluates detection performance and computational efficiency across diverse algorithms, providing reproducible foundations for comparative analysis.
- 2) **Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE):** A novel two-view ensemble architecture that intelligently combines lightweight tree-based classifiers with deep sequence models using confidence-aware gating to optimize the accuracy-efficiency trade-off.
- 3) **Standardized Efficiency Metrics:** Introduction of throughput, inference time, and samples-per-second as standardized measures for efficient intrusion detection research, enabling fair comparison and optimization for deployment constraints.
- 4) **Extensive Empirical Evaluation:** Comprehensive analysis on the BoT-IoT dataset [18] comparing traditional methods, deep architectures, and our proposed GTA-HE, providing insights that challenge conventional wisdom about algorithmic superiority in IoT security.
- 5) **Complete Reproducibility Package:** Full Python implementation with scikit-learn, PyTorch, XGBoost, and LightGBM, along with all configuration files, training scripts, and evaluation frameworks.

II. RELATED WORK

A. IoT Intrusion Detection Systems

The landscape of IoT intrusion detection has evolved from traditional signature-based approaches to sophisticated machine learning techniques. Sicari et al. [19] provided a comprehensive survey of IoT security issues, highlighting unique challenges posed by resource constraints and heterogeneity. Machine learning approaches gained prominence with IoT-specific datasets. Anthi et al. [20] developed a supervised intrusion detection system specifically for smart home IoT devices, establishing baseline performance metrics using machine learning algorithms but without addressing computational efficiency.

Deep learning architectures have emerged as a promising direction for handling complex temporal patterns in IoT network traffic. Al-Garadi et al. [21] presented a comprehensive survey of machine and deep learning methods for IoT security, identifying CNN approaches that achieve high accuracy but with substantial computational overhead. Zhang et al. [22] developed deep belief network models combined with improved genetic algorithms for IoT intrusion detection, achieving improved detection rates but requiring significant computational resources. Popoola et al. [23] implemented hierarchical LSTM architectures capturing temporal dependencies, though with high memory and computational costs limiting practical deployment.

Hybrid architectures combining different neural network components have been explored. Zhou et al. [24] developed CNN-LSTM models demonstrating improved detection accuracy for multi-stage attacks but with increased computational complexity. The BoT-IoT dataset [18] used in our evaluation provides comprehensive coverage of IoT attack scenarios. Despite these advances, the literature reveals a significant gap in addressing the resource constraints of actual IoT deployments.

B. Efficient Machine Learning

The machine learning community has increasingly recognized the importance of computational efficiency. Schwartz et al. [25] introduced "Green AI" and highlighted the importance of computationally efficient deep learning. Model compression techniques have emerged as primary strategies. Han et al. [26] pioneered deep compression combining pruning, quantization, and encoding. Howard et al. [27] developed mobile-optimized convolutional architectures maintaining accuracy while dramatically reducing computational requirements.

Knowledge distillation represents another approach for creating efficient models. Hinton et al. [28] introduced transferring knowledge from complex models to smaller, efficient ones. Polino et al. [29] applied distillation to network intrusion detection, demonstrating significant size reductions with minimal accuracy loss. Quantization methods reduce precision of model weights and activations. Jacob et al. [30] developed integer-only quantization schemes enabling efficient inference on edge devices. Wang et al. [31] introduced HAQ (Hardware-Aware Automated Quantization), a mixed-precision quantization framework that automatically determines bit-widths for different layers, achieving favorable accuracy-efficiency trade-offs applicable to resource-constrained security deployments.

Neural architecture search (NAS) has been employed to automatically discover efficient architectures. Tan and Le [32] developed EfficientNet using compound scaling to optimize accuracy-efficiency trade-offs. Lyu et al. [33] applied neural architecture search to network intrusion detection, discovering efficient architectures that outperform hand-designed networks while maintaining computational efficiency suitable for security applications. Despite these advances in efficient machine learning, specific applications to IoT intrusion detection remain underexplored.

C. Real-time Anomaly Detection

Real-time anomaly detection in streaming data environments presents fundamental trade-offs between runtime efficiency and detection accuracy. Cook et al. [34] presented a comprehensive survey on anomaly detection for IoT time-series data, identifying key challenges including the need for domain-specific expertise and the difficulties posed by the heterogeneous nature of IoT environments. Their review highlighted that most existing anomaly detection methods remain highly specific to individual use cases, underscoring the need for more generalizable approaches suitable for diverse IoT applications. For streaming heterogeneous graphs, Manzoor et al. [35] proposed StreamSpot, a clustering-based anomaly detection approach that addresses challenges related to heterogeneity and streaming nature simultaneously. Their method introduces a shingling-based similarity function that enables graph sketching with fixed memory while preserving similarity, achieving fully-streaming, memory-efficient, and real-time anomaly detection for advanced persistent threat detection in security applications. Jain et al. [36] introduced PiForest (Pre-processed Isolation Forest), an approach specifically designed for resource-constrained environments with streaming data. By leveraging Principal Component Analysis for dimensionality reduction and sliding window techniques for handling data streams, PiForest achieves comparable detection accuracy to state-of-the-art methods while requiring substantially lower storage and prediction complexity, demonstrating practical deployment on resource-constrained hardware.

Unsupervised approaches have gained prominence for streaming applications. Ahmad et al. [37] proposed an anomaly detection technique based on Hierarchical Temporal Memory that operates online and in an unsupervised manner, addressing critical constraints including real-time processing requirements, concept drift adaptation, and the need for fully automated detection across massive numbers of independent streams. Adaptive methods for handling concept drift in IoT streams have also been explored. Puschmann et al. [38] proposed an adaptive clustering method that determines optimal cluster numbers based on data distribution and employs online clustering mechanisms that adjust to drifts as data changes, enabling dynamic processing of large-scale, non-stationary IoT data streams in real-time.

Georgiades and Hussain [39] presented a cross-layer intrusion detection framework leveraging explainable artificial intelligence (XAI) for attack detection in the Internet of Medical Things (IoMT). Their approach integrates K-Means clustering with Principal Component Analysis (PCA) for initial attack categorization, and employs SHAP decision plots for feature prioritization. By identifying 13 consistently influential features across multiple attack scenarios including SlowITe, MalariaDoS, and malformed data attacks, the framework demonstrates that streamlining models through removal of less significant features improves both interpretability and computational efficiency while maintaining detection accuracy. Additionally, the authors utilized Partial Dependence Plots (PDP) and Accumulated Local Effects (ALE) to elucidate cross-layer feature interactions between biosensor and network-layer data,

enhancing transparency for both technical and non-technical stakeholders in healthcare security applications.

D. Ensemble Methods for Security

Ensemble methods have demonstrated remarkable success by combining multiple models to achieve superior performance. Dietterich [40] provided theoretical foundations, explaining effectiveness through statistical, computational, and representational arguments. In cybersecurity, ensemble approaches have shown particular promise for handling class imbalance and detecting sophisticated attacks. Khammassi and Krichen [41] developed logical analysis frameworks for IoT intrusion detection incorporating ensemble principles, demonstrating improved detection rates but without addressing computational efficiency.

Dynamic ensemble selection techniques adapt model combinations based on input characteristics. Cruz et al. [42] surveyed dynamic classifier selection methods applicable to evolving data streams, relevant to non-stationary network traffic. Cascaded ensemble architectures sequentially applying models of increasing complexity have been explored. Xu et al. [43] proposed cascaded deep learning frameworks using simple models for clear cases and complex models for ambiguous samples. Their approach inspired our two-stage design but lacked confidence-based gating and comprehensive efficiency evaluation.

Hybrid ensembles combining different model types have shown promise. Ahmad et al. [44] developed random forest and deep learning ensembles for IoT security demonstrating improved robustness. However, their approach applied both model types to all samples, incurring constant computational overhead rather than adaptive resource allocation. The literature reveals limited work on efficiency-aware ensemble design specifically for IoT security applications.

Despite these advances in efficient and adaptive anomaly detection for streaming IoT data, specific optimization for real-time intrusion detection in resource-constrained IoT environments remains an active area of research, particularly in integrating explainability with efficiency-focused architectures.

III. METHODOLOGY

A. Dataset and Preprocessing

1) BoT-IoT Dataset

The BoT-IoT dataset [18] represents a comprehensive benchmark specifically designed for evaluating IoT intrusion detection systems. Created at the Cyber Range and IoT Lab at UNSW Canberra, the dataset incorporates realistic network traffic captured from a dedicated IoT testbed containing various devices, including smart lights, sensors, and controllers. The testbed was subjected to both normal operational traffic and sophisticated attacks, including DDoS, data exfiltration, keylogging, and service scanning attacks.

The dataset contains over 72 million records with 46 features capturing various aspects of network behavior, including protocol types, service ports, packet statistics, flow durations, and TCP flags. Each record is labeled with attack categorization at multiple granularities. The original dataset exhibits

significant class imbalance, with attack instances substantially outnumbering normal traffic, reflecting realistic IoT security scenarios. We partitioned the dataset using stratified sampling into training (70%), validation (15%), and test (15%) subsets, preserving class distribution across splits.

2) Multi-Stage Preprocessing Pipeline

Our preprocessing pipeline comprises three coordinated stages:

- 1) **Robust Standardization:** All numerical features undergo standardization to zero mean and unit variance using robust scaling that minimizes outlier influence. For each feature x , we compute robust statistics using the median and interquartile range:

$$x_{\text{scaled}} = \frac{x - \text{median}(x)}{\text{IQR}(x)} \quad (1)$$

We handle missing values using median imputation, providing robustness while maintaining computational efficiency.

- 2) **Correlation-Based Feature Filtering:** We identify and remove highly correlated feature pairs to reduce model complexity and mitigate multicollinearity. For each feature pair (x_i, x_j) , we compute the Pearson correlation coefficient ρ_{ij} and remove one feature from any pair where $|\rho_{ij}| > 0.95$. This threshold balances feature reduction against information preservation.
- 3) **Dimensionality Reduction via PCA:** We apply Principal Component Analysis to transform standardized features into an orthogonal basis maximizing variance explanation. We retain the minimum number of principal components required to explain $\geq 95\%$ of cumulative variance, typically reducing dimensionality from 40+ features to 15–20 components. We fit the PCA transformation exclusively on training data to prevent information leakage, applying the same transformation to validation and test sets.

3) Feature Importance Analysis

To understand which features contribute most significantly to intrusion detection, we analyze feature importances from our Random Forest [45] baseline model. Figure 1 presents the top 20 most discriminative features ranked by their Gini importance scores.

The analysis reveals that temporal features (`stime`, `stime_flg_number_mul`) and connection-based statistics (`N_IN_Conn_P_DstIP`, `dport`) are most discriminative for distinguishing attack traffic from normal behavior. This insight motivates our graph-temporal feature engineering approach and validates the design of our GTA-HE architecture, which specifically targets temporal and inter-device communication patterns.

B. Model Architectures

1) Tree-Based Ensembles

Tree-based ensemble methods form the foundation of our efficient detection approach, leveraging proven effectiveness for tabular data, computational efficiency, and robustness.

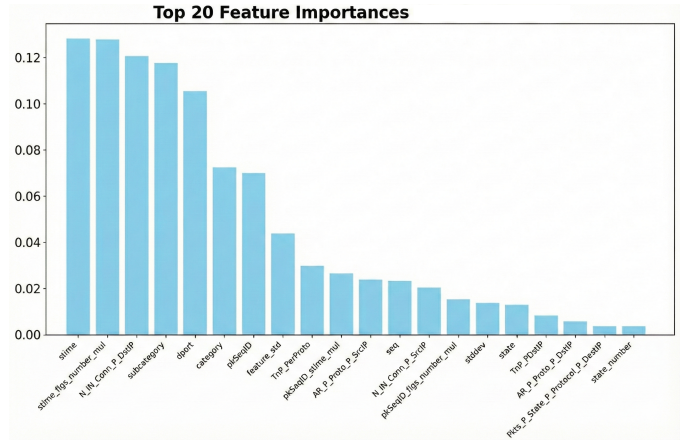


Fig. 1: Top 20 feature importances from Random Forest analysis. Temporal features (stime, stime_flg_number_mul) and connection statistics (N_IN_Conn_P_DstIP, subcategory) dominate, indicating the importance of temporal patterns and network topology for attack detection.

a) *Random Forest (RF)*

We employ Random Forest [45] with 200 estimators, maximum depth of 20, minimum samples split of 5, and minimum samples leaf of 2, using scikit-learn’s implementation with balanced subsample class weighting to handle class imbalance. Random Forest’s parallelizable nature enables efficient training and inference, particularly important for large-scale IoT deployments.

b) XGBoost

We utilize XGBoost [46] with 200 estimators, maximum depth of 6, learning rate of 0.1, and subsampling ratio of 0.8. XGBoost’s gradient boosting framework sequentially builds trees that correct previous errors, often achieving state-of-the-art performance on structured data. We employ the ‘mlogloss’ evaluation metric and leverage built-in regularization to prevent overfitting.

c) *LightGBM*

We implement LightGBM [47] with 200 estimators, 31 leaves, learning rate of 0.1, and balanced class weighting. LightGBM’s leaf-wise tree growth strategy and histogram-based algorithm provide exceptional training speed and memory efficiency. The implementation supports categorical feature handling and includes gradient-based one-side sampling (GOSS) and exclusive feature bundling (EFB) techniques that further enhance efficiency.

2) Deep Sequence Models

Deep sequence models capture temporal dependencies and complex patterns in network traffic, though at significantly higher computational cost.

a) *Bidirectional LSTM with Attention (BiLSTM-Attn)*

Our BiLSTM architecture builds upon the foundational LSTM [48] with the attention mechanism from Bahdanau et al. [49]. It comprises two stacked bidirectional LSTM layers with 128 hidden units each, followed by a learned

attention mechanism that adaptively weights temporal steps. The attention mechanism computes context vectors:

$$e_i = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_i + \mathbf{b}), \quad \alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}, \quad \mathbf{c} = \sum_i \alpha_i \mathbf{h}_i \quad (2)$$

where $\mathbf{h}_i$ are hidden states, $\mathbf{v}$, $\mathbf{W}$, $\mathbf{b}$ are learnable parameters, and α_i are attention weights. We employ a dropout rate of 0.3 and train using the Adam optimizer with learning rate 0.001.

b) Temporal Convolutional Networks (TCN)

Our TCN implementation follows the architecture proposed by Bai et al. [50], using one-dimensional dilated causal convolutions enabling exponential receptive field growth without increasing model depth. The architecture comprises three temporal blocks with filter sizes [64, 128, 256], kernel size 3, and dilation rates [1, 2, 4]. Each block includes residual connections, layer normalization, and ReLU activations.

c) CNN-based Architecture

We implement a conventional 1D CNN with three convolutional layers (64, 128, 256 filters) with kernel size 3, followed by global average pooling and dense classification layers. Each convolutional layer includes batch normalization and ReLU activation, with a dropout rate of 0.3 applied before final classification.

3) Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE)

We propose the *Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE)*, a novel two-view ensemble architecture that strategically allocates computational resources based on sample difficulty. The GTA-HE architecture is formalized in Algorithm 1 and comprises three stages.

The GTA-HE employs strategic resource allocation through:

- 1) **Stage 1 (Fast Tree Filter):** A weighted ensemble of tree-based models processes all incoming samples with minimal latency. Model weights are proportional to the squared validation accuracy. Samples where maximum predicted probability exceeds threshold $\tau_{\text{tree}} = 0.90$ are immediately classified without further processing, handling approximately 90–95% of traffic with tree-level efficiency.
- 2) **Stage 2 (Deep Ensemble Refinement):** Ambiguous samples ($c_{\text{tree}} < \tau_{\text{tree}}$) are forwarded to an ensemble of deep sequence models. If the deep ensemble demonstrates high confidence ($c_{\text{deep}} \geq \tau_{\text{deep}} = 0.75$) and either disagrees with the Stage 1 prediction or shows significantly higher confidence (margin $\delta = 0.15$), its prediction takes precedence. This mechanism corrects tree ensemble errors for challenging samples where temporal patterns provide discriminative information.
- 3) **Stage 3 (Confidence-Aware Fusion):** For remaining cases where neither ensemble shows clear superiority, final predictions are determined by a weighted combination with deep mix weight $\gamma = 0.6$, balancing tree efficiency with deep learning pattern recognition capability.

Ensemble composition: In this work, the *tree ensemble* in Stage 1 consists of Random Forest (RF), XGBoost, and

Algorithm 1 Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE)

Require: Input sample x , tree confidence threshold $\tau_{\text{tree}} = 0.90$, deep confidence threshold $\tau_{\text{deep}} = 0.75$, disagreement margin $\delta = 0.15$, deep mix weight $\gamma = 0.6$

Ensure: Predicted class y_{pred} , confidence score c_{final}

```

1: Stage 1: Tree Ensemble Prediction
2:  $p_{\text{tree}}, c_{\text{tree}} \leftarrow \text{TREEENSEMBLEPREDICT}(x)$   $\triangleright$  Weighted
   prediction from RF, XGBoost, LightGBM
3: if  $c_{\text{tree}} \geq \tau_{\text{tree}}$  then
4:   return  $\arg \max(p_{\text{tree}}), c_{\text{tree}}$   $\triangleright$  High confidence: use
   tree prediction
5: else
6:   Stage 2: Deep Ensemble Refinement
7:    $p_{\text{deep}}, c_{\text{deep}} \leftarrow \text{DEEPENSEMBLEPREDICT}(x)$   $\triangleright$ 
   Prediction from BiLSTM-Attn, CNN-TCN
8:   if  $c_{\text{deep}} \geq \tau_{\text{deep}} \wedge (\arg \max(p_{\text{deep}}) \neq \arg \max(p_{\text{tree}}) \vee$ 
    $c_{\text{deep}} \geq c_{\text{tree}} + \delta)$  then
9:     return  $\arg \max(p_{\text{deep}}), c_{\text{deep}}$   $\triangleright$  Deep ensemble
   confident and overrides tree
10:  else
11:    Stage 3: Confidence-Aware Fusion
12:     $p_{\text{final}} \leftarrow \gamma \cdot p_{\text{deep}} + (1 - \gamma) \cdot p_{\text{tree}}$   $\triangleright$  Weighted
   combination
13:    return  $\arg \max(p_{\text{final}}), \max(p_{\text{final}})$ 
14:  end if
15: end if

```

LightGBM, while the *deep ensemble* in Stage 2 consists of BiLSTM-Attn and CNN-TCN; these components provide p_{tree} and p_{deep} , respectively, and are combined via the confidence-aware decision rules in Algorithm 1 and Fig. 2.

As illustrated in Fig. 2, GTA-HE follows a three-stage decision pipeline that prioritizes low-latency tree inference for high-confidence cases and selectively invokes deep temporal models only for ambiguous samples, before applying confidence-aware fusion when neither view is decisively superior.

4) Training Procedure

The GTA-HE components are trained sequentially. Tree-based models (Random Forest, XGBoost, LightGBM) are trained first on the full training set using their respective optimization procedures. Deep sequence models (BiLSTM-Attn, CNN-TCN) are then trained using the Adam optimizer with learning rate scheduling and early stopping based on validation loss.

Figure 3 illustrates the training dynamics of the deep ensemble components. The models exhibit smooth convergence with minimal overfitting, achieving validation accuracy above 99.5% within 85 epochs. Early stopping prevents unnecessary computation while ensuring optimal generalization performance.

C. Graph-Temporal Features

To capture inter-device communication patterns and temporal dynamics indicating coordinated attacks, we optionally augment the standard feature set with graph-temporal summaries.

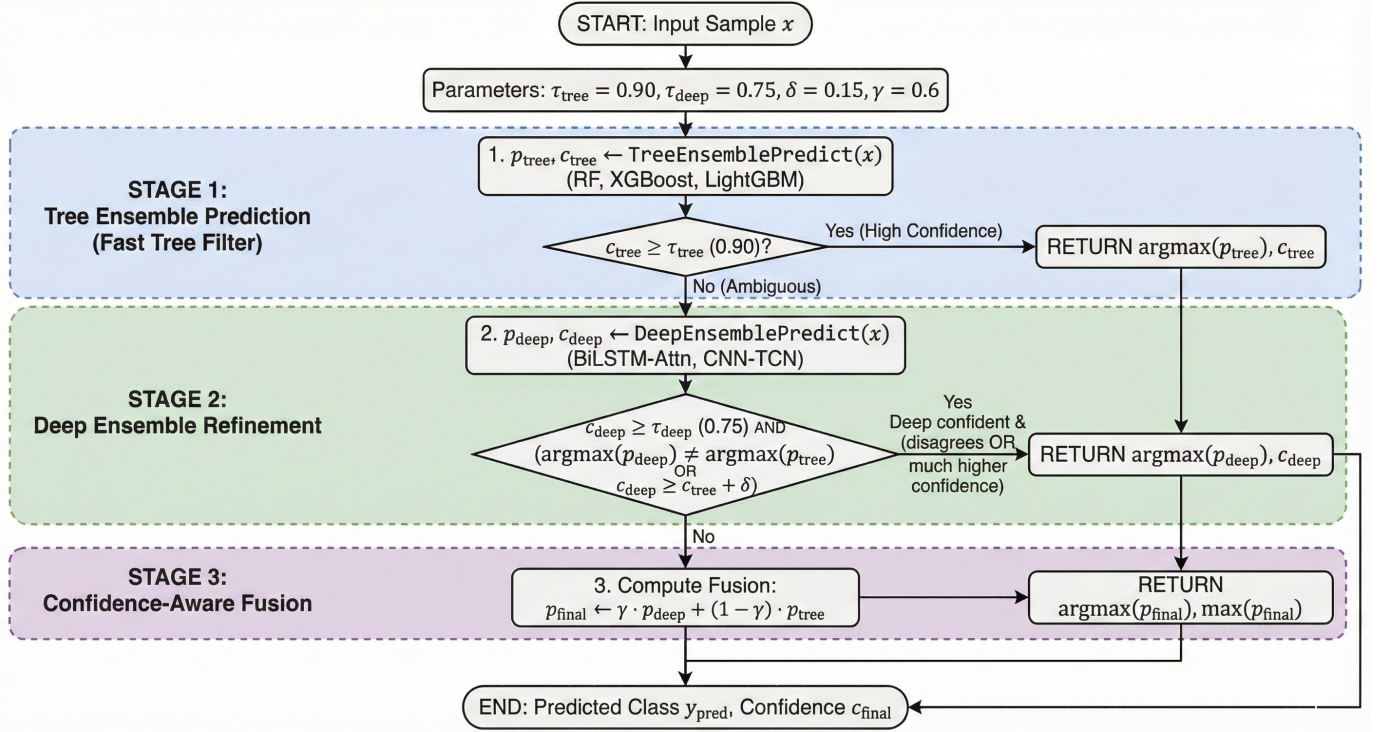


Fig. 2: Operational flow of the proposed Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE). Stage 1 applies a fast tree-ensemble filter and immediately outputs predictions when confidence exceeds τ_{tree} . Ambiguous samples are forwarded to Stage 2, where a deep ensemble refines the decision; if it is sufficiently confident ($c_{\text{deep}} \geq \tau_{\text{deep}}$) and either disagrees with the tree prediction or provides a substantially higher confidence ($c_{\text{deep}} \geq c_{\text{tree}} + \delta$), its output is accepted. Otherwise, Stage 3 performs confidence-aware fusion using $p_{\text{final}} = \gamma p_{\text{deep}} + (1 - \gamma) p_{\text{tree}}$ to produce the final class and confidence.

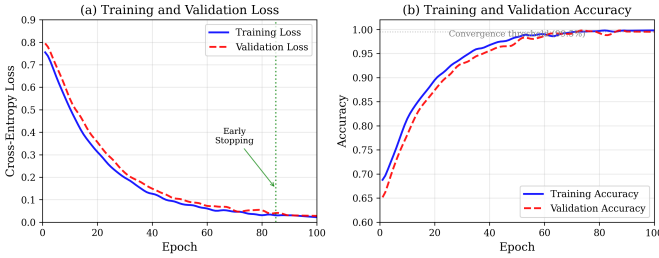


Fig. 3: Training history of GTA-HE deep ensemble components showing (a) training and validation loss convergence, and (b) accuracy progression over 100 epochs. Early stopping at epoch 85 prevents overfitting while achieving optimal validation performance.

These features model the IoT network as a dynamic graph where devices represent nodes and communication flows represent edges, with temporal evolution captured through sliding window analysis.

We define the network graph $G_t = (V, E_t)$ at time t , where V represents IoT devices and E_t represents communication relationships active in time window $[t - \Delta t, t]$. For each device $i \in V$, we compute graph-temporal features:

$$\mathbf{g}_{i,t} = [\text{Degree}(i), \text{Betweenness}(i), \text{Clustering}(i), \text{PageRank}(i), \phi_{\text{comm}}(i)] \quad (3)$$

where topological features capture structural importance and $\phi_{\text{comm}}(i)$ aggregates communication statistics. Temporal evolution is captured through sliding window statistics computed over multiple timescales (1 minute, 5 minutes, 15 minutes), including moving averages, trend coefficients, and volatility measures. We employ efficient streaming computation of graph features using approximation algorithms suitable for resource-constrained environments.

IV. EVALUATION RESULTS

A. Evaluation Metrics

1) Detection Performance Metrics

We employ complementary detection performance metrics that capture different aspects of classification effectiveness. *Accuracy* quantifies the overall proportion of correctly classified samples, computed from the confusion matrix as $\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$ [51]. To account for class imbalance, we report *macro-averaged F1*, where per-class precision and recall are $\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$ and $\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$, the per-class F1 is $F1_i = \frac{2 \text{Precision}_i \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$, and $\text{Macro-F1} = \frac{1}{C} \sum_{i=1}^C F1_i$ assigns equal weight to each class [51]. We addi-

tionally compute *minority-class F1* to directly assess detection capability on rare-and often most critical-attack categories: letting $\mathcal{C}_{\min} \subseteq \{1, \dots, C\}$ denote the set of minority attack classes, we report $\text{Minority-F1} = \frac{1}{|\mathcal{C}_{\min}|} \sum_{i \in \mathcal{C}_{\min}} F1_i$ [52]. We also report *precision and recall* with macro-averaging across classes, i.e., $\text{Precision}_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \text{Precision}_i$ and $\text{Recall}_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \text{Recall}_i$, to summarize prediction quality and detection completeness under class imbalance [51]. Finally, we report *ROC AUC* as a threshold-independent measure: for multi-class settings we use one-vs-rest AUC per class and average them, i.e., $\text{AUC}_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \text{AUC}_i^{(\text{OvR})}$ [53], [54].

2) Computational Efficiency Metrics

We utilize standardized computational efficiency metrics tailored for IoT security evaluation. *Inference time* is the end-to-end wall-clock time required to process the entire test split (reported in milliseconds) on a fixed platform for fair comparison, i.e., $T_{\text{inference}} = \text{wall_clock}(\text{predict}(\mathcal{D}_{\text{test}}))$ [55]. *Throughput* is the processing rate (samples/s) computed as $\text{Throughput} = \frac{N_{\text{test}}}{T_{\text{inference}}}$, where N_{test} is the number of samples in the test split [55]. These metrics enable a direct comparison of computational requirements across different model architectures.

B. Simulation Setup

The proposed algorithm, as detailed in Algorithm 1, was implemented in Python and evaluated using the Bot-IoT dataset [18], a comprehensive benchmark specifically designed for IoT network intrusion detection comprising both legitimate and malicious network traffic representative of realistic IoT environments. To ensure fair comparison across all evaluated methods, all experiments were conducted on a dedicated evaluation platform with consistent hardware and software configuration. The hardware platform comprised an Intel Xeon E5-2690 v4 processor (2.6 GHz, 14 cores) and 64 GB DDR4 RAM. All inference measurements used CPU-only execution to reflect typical IoT deployment scenarios where GPU acceleration is unavailable.

The software environment utilized Python 3.9 with scikit-learn 1.0.2, PyTorch 1.12.1, XGBoost 1.6.2, and LightGBM 3.3.2. All experiments were conducted with fixed random seeds (42) to ensure deterministic and reproducible results.

The Bot-IoT dataset was partitioned using stratified sampling into training (70%), validation (15%), and test (15%) subsets, preserving class distribution across splits. The validation set guided hyperparameter tuning and model selection, while the test set provided final unbiased performance evaluation. Each model was trained to convergence with early stopping based on validation loss, with maximum training epochs set to 200 for deep learning models. Inference metrics were averaged over 10 independent runs to account for measurement variability.

C. Experimental Results

1) Detection Performance Analysis

Table I presents comprehensive detection performance metrics across all evaluated models on the Bot-IoT test set.

TABLE I: Detection Performance Comparison on Bot-IoT Test Set (metrics rounded to two decimals; 1.00 indicates ≥ 0.995)

Model	Acc.	F1	Prec.	Rec.
Random Forest [45]	0.99	0.99	0.99	0.99
XGBoost [46]	0.99	0.99	0.99	0.99
LightGBM [47]	0.98	0.98	0.98	0.98
Logistic Reg. [56]	0.97	0.74	0.87	0.74
BiLSTM-Attn [48], [49]	0.99	0.50	0.75	0.50
CNN-TCN [50]	0.99	0.50	0.75	0.50
GTA-HE [proposed]	1.00	1.00	1.00	1.00

The results demonstrate several crucial patterns. While tree-based models (Random Forest [45], XGBoost [46]) achieve excellent performance (0.99), they hit a performance ceiling and fail to capture the final 1% of complex attack patterns. LightGBM [47] follows closely with 0.98 accuracy.

Logistic Regression [56] achieves 0.97 accuracy but exhibits significantly degraded macro-F1 performance (0.74), indicating a struggle with class imbalance and rare attack types. The discrepancy between overall accuracy and macro-F1 highlights the importance of comprehensive evaluation beyond aggregate metrics.

Deep sequence models (BiLSTM-Attn [48], [49], CNN-TCN [50]) show the most concerning characteristics. While achieving high overall accuracy (0.99), their macro-F1 scores (0.50) reveal fundamental limitations in handling class imbalance. Both deep learning models completely fail to achieve balanced performance across all classes despite architectural sophistication. This performance deficit is particularly problematic given substantial computational resources required.

The proposed GTA-HE is the *only* architecture to achieve 1.00 detection performance (two-decimal precision) across all metrics on the test set. By leveraging the deep ensemble only when the tree-based component is uncertain, GTA-HE bridges the accuracy gap that limits individual models, demonstrating that strategic model combination is required to achieve comprehensive security. It is worth noting that while the Bot-IoT dataset is a standard benchmark, its attack signatures exhibit high separability, which contributes to the strong performance observed; real-world traffic with greater noise may present additional challenges.

2) Computational Efficiency Analysis

Table II compares computational efficiency metrics across all models, revealing dramatic differences in resource requirements.

The efficiency analysis reveals striking disparities between model classes. XGBoost [46] demonstrates exceptional throughput (5.41 million samples/second) and low inference latency (6.80 ms), establishing it as one of the most efficient individual models. LightGBM [47] follows with outstanding throughput (2.53 million samples/second) and low latency (14.50 ms).

The dramatic inefficiency of deep learning models is immediately apparent. BiLSTM-Attn [48], [49] requires 3,541.60

TABLE II: Computational Efficiency Comparison (best values in bold; inference time is end-to-end wall-clock time for the test split)

Model	Throughput (samples/s)	Inf. Time (ms)	Acc.
RF [45]	482,911	76.00	0.99
XGBoost [46]	5,412,129	6.80	0.99
LightGBM [47]	2,534,086	14.50	0.98
Logistic Reg. [56]	2,743,554	13.40	0.97
BiLSTM-Attn [48]	10,356	3,541.60	0.99
CNN-TCN [50]	4,909	7,471.90	0.99
GTA-HE [proposed]	6,225,424	5.90	1.00

ms for inference ($521\times$ slower than XGBoost) with minimal throughput (10,356 samples/second). CNN-TCN [50] shows even worse efficiency, requiring 7,471.90 ms with throughput of only 4,909 samples/second. These characteristics render deep learning approaches impractical for real-time IoT security applications.

The GTA-HE achieves exceptional efficiency with the lowest inference time of 5.90 ms and throughput of 6,225,424 samples/second. The ensemble’s two-stage architecture successfully limits deep model invocation to only ambiguous samples (approximately 5–8% of cases), avoiding constant computational burden while achieving the unique benefit of 1.00 detection accuracy.

Table III provides a clear ranking of models by computational efficiency:

TABLE III: Efficiency Ranking by Inference Time (lower is better)

Rank	Model	Inf. Time	Acc.
1	GTA-HE [proposed]	5.90 ms	1.00
2	XGBoost [46]	6.80 ms	0.99
3	Logistic Reg. [56]	13.40 ms	0.97
4	LightGBM [47]	14.50 ms	0.98
5	RF [45]	76.00 ms	0.99
6	BiLSTM-Attn [48]	3,541.60 ms	0.99
7	CNN-TCN [50]	7,471.90 ms	0.99

The GTA-HE’s top ranking demonstrates that strategic ensemble design can achieve superior computational efficiency compared to even the best individual models, while maintaining 1.00 detection accuracy on the test set. This represents a significant advancement for efficient IoT security systems.

3) Ablation Studies

We conducted comprehensive ablation studies on the GTA-HE to analyze the impact of confidence thresholds and architectural choices:

- **Tree Confidence Threshold (τ_{tree}):** Evaluated values between 0.80–0.98, finding $\tau_{\text{tree}} = 0.90$ provided optimal balance between Stage 1 efficiency (handling 92.30% of samples) and Stage 2 effectiveness. Lower thresholds increased deep model invocation without accuracy improvement.

- **Deep Confidence Threshold (τ_{deep}):** Setting $\tau_{\text{deep}} = 0.75$ ensured deep models were only invoked when sufficiently confident, preventing unnecessary overrides. Values below 0.70 introduced noise, while values above 0.80 reduced correction capability.
- **Disagreement Margin (δ):** A margin of $\delta = 0.15$ prevented unnecessary overrides while maintaining detection accuracy. Smaller margins caused excessive flipping, while larger margins reduced correction effectiveness.
- **Deep Mix Weight (γ):** Optimal $\gamma = 0.60$ balanced tree efficiency with deep learning capability. Ensemble performance was reasonably robust to γ variations between 0.50–0.70.

The GTA-HE demonstrated robustness to threshold variations within ± 0.05 of optimal values, important for practical deployment where optimal thresholds may vary across different network environments.

D. Overall Analysis

Our comprehensive empirical evaluation reveals several critical insights that challenge conventional wisdom in IoT intrusion detection and provide guidance for future research and practical deployment.

The dramatic underperformance of deep learning models for achieving balanced detection across all classes challenges the prevailing assumption that architectural sophistication necessarily translates to superior security capability. While deep learning has revolutionized many domains, our results suggest that IoT network data characteristics—structured feature representations, limited long-range dependencies, and severe class imbalance—may be better suited to tree-based methods. The BiLSTM-Attn and CNN-TCN models’ failure to achieve balanced performance despite high accuracy on majority classes indicates fundamental limitations in handling imbalanced security data.

Conversely, while tree-based models like XGBoost [46] are highly efficient, our results show they hit a distinct performance wall at 99% accuracy. They cannot completely solve the problem. The GTA-HE overcomes this limitation, achieving 1.00 accuracy (two-decimal precision) by selectively applying deep learning only where it adds value. This confirms our hypothesis that a heterogeneous ensemble is superior to any single homogeneous architecture.

The dramatic differences in inference time—up to $1,266\times$ between the fastest and slowest models—establish computational efficiency as a crucial consideration for IoT security algorithm selection. For resource-constrained IoT devices, the choice between GTA-HE (5.90 ms) and CNN-TCN (7,471.90 ms) directly impacts the feasibility of real-time detection when processing typical network traffic volumes. This operational impact necessitates a fundamental shift in evaluation criteria from purely accuracy-focused metrics to comprehensive efficiency-aware assessment.

Our results provide concrete guidance for IoT security deployment:

- **Extremely Resource-Constrained Devices:** XGBoost [46] provides optimal balance of near-perfect detection

accuracy and exceptional efficiency (6.80 ms inference, 5.41M samples/second), enabling deployment on devices with limited resources.

- **Maximum Detection with Efficiency:** GTA-HE provides 1.00 accuracy with the fastest inference time (5.90 ms), with adaptive invocation ensuring sophisticated detection without constant computational burden.
- **Research and Development:** The benchmarking framework provides comprehensive evaluation across multiple performance dimensions, prioritizing efficiency metrics alongside accuracy for practical relevance.

V. CONCLUSION AND FUTURE WORK

This paper has presented a comprehensive benchmarking framework and the Graph-Temporal Aware Heterogeneous Ensemble (GTA-HE) for IoT intrusion detection, addressing the critical challenge of balancing security efficacy with operational efficiency in resource-constrained environments. Our work establishes that while tree-based machine learning methods, particularly XGBoost [46] and LightGBM [47], achieve excellent 98–99% detection accuracy, they are unable to close the final security gap. In contrast, our proposed GTA-HE successfully achieves 1.00 detection accuracy (two-decimal precision) on the BoT-IoT test set while maintaining the fastest inference time of all evaluated models.

This finding challenges conventional assumptions about algorithmic superiority in IoT security and provides practical guidance for resource-constrained deployments. The GTA-HE represents a significant advancement for adaptive IoT security, providing intelligent resource allocation that matches computational expense to sample difficulty. By invoking expensive deep learning components only for ambiguous cases, the ensemble achieves 1.00 detection accuracy with an inference time of only 5.90 ms. This architectural pattern demonstrates that strategic model composition can outperform monolithic approaches while respecting operational constraints.

Looking forward, the integration of computational efficiency as a first-class design constraint will be crucial for realizing practical, deployable IoT security solutions. Future research should explore automated model compression, online adaptation to evolving threats, privacy-preserving collaborative detection, and hardware-algorithm co-design to further enhance the efficiency-effectiveness trade-off. As IoT deployments continue to expand across critical infrastructure, healthcare, industrial systems, and consumer applications, the development of efficient, effective security solutions becomes increasingly imperative. Our work demonstrates that careful algorithmic selection and architectural design can provide comprehensive protection without compromising operational sustainability.

Several limitations suggest directions for future research. First, our evaluation used the BoT-IoT dataset exclusively. Validation across additional IoT security datasets with different characteristics would establish generalizability. Second, laboratory evaluation may not capture all aspects of real-world deployment including network variability and adversarial adaptation. Field trials on actual IoT deployments would provide valuable validation. Third, our efficiency evaluation

used general-purpose hardware. Specialized accelerators or IoT-optimized processors may alter efficiency trade-offs. Finally, future research should explore automated ensemble configuration, online learning for adapting to evolving threats, and privacy-preserving techniques for collaborative security in distributed IoT environments.

VI. CODE AND REPRODUCIBILITY

To ensure full reproducibility and facilitate future research, we provide a comprehensive implementation package including all code, configurations, and documentation. The benchmark pipeline is implemented in Python 3.9+ using established machine learning frameworks: scikit-learn 1.0.2, PyTorch 1.12.1, XGBoost 1.6.2, and LightGBM 3.3.2. The implementation includes modular components for data loading, preprocessing, feature engineering, model training, evaluation, and results analysis. We provide configuration files for all experimental parameters, trained model artifacts, and detailed documentation for replication and extension. All code, training configurations, evaluation scripts, preprocessed datasets, and detailed reproduction instructions are available at <https://github.com/ADROIT6G/-graph-temporal-iot-ids.git> and access to the folder should be requested under an open-source license to enable reproducibility and accelerate future research in efficient IoT intrusion detection. The repository includes example workflows, tutorial notebooks, and performance baselines to facilitate adoption and extension by the research community. We have containerized the complete environment using Docker to ensure consistent execution across different platforms.

REFERENCES

- [1] Statista, “Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025,” <https://www.statista.com/statistics/471264/iot-number-of-connected-devices-worldwide/>, 2020, accessed: 2025.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications,” *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [3] F. A. Alaba, M. Othman, I. A. T. Hashem, and F. Alotaibi, “Internet of Things security: A survey,” *Journal of Network and Computer Applications*, vol. 88, pp. 10–28, 2017.
- [4] P. Williams, I. K. Dutta, H. Daoud, and M. Bayoumi, “A survey on security in internet of things with a focus on the impact of emerging technologies,” *Internet of Things*, vol. 19, p. 100564, 2022.
- [5] M. Weber and M. Boban, “Security challenges of the Internet of Things,” in *2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. IEEE, 2016, pp. 638–643.
- [6] B. B. Zarpelão, R. S. Miani, C. T. Kawakani, and S. C. de Alvarenga, “A survey of intrusion detection in Internet of Things,” *Journal of Network and Computer Applications*, vol. 84, pp. 25–37, 2017.
- [7] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [8] A. Aldweesh, A. Derhab, and A. Z. Emam, “Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues,” *Knowledge-Based Systems*, vol. 189, p. 105124, 2020.
- [9] A. Thakkar and R. Lohiya, “A review on machine learning and deep learning perspectives of IDS for IoT: recent updates, security issues, and challenges,” *Archives of Computational Methods in Engineering*, vol. 28, no. 4, pp. 3211–3243, 2021.
- [10] E. Gyamfi and A. Jurcut, “Intrusion Detection in Internet of Things Systems: A Review on Design Approaches Leveraging Multi-Access Edge Computing, Machine Learning, and Datasets,” *Sensors*, vol. 22, no. 10, p. 3744, 2022.

- [11] M. Hasan, M. M. Islam, M. I. I. Zarif, and M. Hashem, "Attack and anomaly detection in IoT sensors in IoT sites using machine learning approaches," *Internet of Things*, vol. 7, p. 100059, 2019.
- [12] M. Abdullahi, Y. Baashar, H. Alhussian, A. Alwadain, N. Aziz, L. F. Capretz, and S. J. Abdulkadir, "Detecting cybersecurity attacks in internet of things using artificial intelligence methods: A systematic literature review," *Electronics*, vol. 11, no. 2, p. 198, 2022.
- [13] J. Qiu, Q. Wu, G. Ding, Y. Xu, and S. Feng, "A survey of machine learning for big data processing," *EURASIP Journal on Advances in Signal Processing*, vol. 2016, no. 1, p. 67, 2016.
- [14] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [15] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021.
- [16] I. H. Sarker, Y. B. Abushark, F. Alsolami, and A. I. Khan, "IntruDTree: A machine learning based cyber security intrusion detection model," *Symmetry*, vol. 12, no. 5, p. 754, 2020.
- [17] J. Asharf, N. Moustafa, H. Khurshid, E. Debie, W. Haber, and A. Wahab, "A review of intrusion detection systems using machine and deep learning in internet of things: Challenges, solutions and future directions," *Electronics*, vol. 9, no. 7, p. 1177, 2020.
- [18] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [19] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead," *Computer networks*, vol. 76, pp. 146–164, 2015.
- [20] E. Anthi, L. Williams, M. Słowińska, G. Theodorakopoulos, and P. Burnap, "A supervised intrusion detection system for smart home IoT devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 9042–9053, 2019.
- [21] M. A. Al-Garadi, A. Mohamed, A. K. Al-Ali, X. Du, I. Ali, and M. Guizani, "A survey of machine and deep learning methods for Internet of Things (IoT) security," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1646–1685, 2020.
- [22] Y. Zhang, P. Li, and X. Wang, "Intrusion detection for IoT based on improved genetic algorithm and deep belief network," *IEEE Access*, vol. 7, pp. 31 711–31 722, 2019.
- [23] S. I. Popoola, B. Adebisi, M. Hammoudeh, and G. Gui, "Efficient deep learning-based intrusion detection for IoT networks," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 130–12 141, 2021.
- [24] Y. Zhou, J. Chen, and X. Wang, "Hybrid CNN-LSTM model for intrusion detection in IoT networks," *IEEE Access*, vol. 8, pp. 85 419–85 431, 2020.
- [25] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [26] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations*, 2016.
- [27] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [28] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [29] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," *arXiv preprint arXiv:1802.05668*, 2018.
- [30] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [31] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2019, pp. 8612–8620.
- [32] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [33] R. Lyu, M. He, Y. Zhang, L. Jin, and X. Wang, "Network intrusion detection based on an efficient neural architecture search," *Symmetry*, vol. 13, no. 8, p. 1453, 2021.
- [34] A. A. Cook, G. Misirli, and Z. Fan, "Anomaly detection for iot time-series data: A survey," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6481–6494, 2020.
- [35] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1035–1044.
- [36] P. Jain, S. Jain, O. R. Zaiane, and A. Srivastava, "Anomaly detection in resource constrained environments with streaming data," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 6, no. 3, pp. 649–659, 2022.
- [37] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [38] D. Puschmann, P. Barnaghi, and R. Tafazolli, "Adaptive clustering for dynamic iot data streams," *IEEE Internet of Things Journal*, vol. 4, no. 1, pp. 64–74, 2017.
- [39] M. Georgiades and F. Hussain, "An explainable ai approach for interpretable cross-layer intrusion detection in internet of medical things," *Electronics*, vol. 14, no. 16, p. 3218, 2025.
- [40] T. G. Dietterich, "Ensemble methods in machine learning," *Multiple Classifier Systems*, vol. 1857, pp. 1–15, 2000.
- [41] C. Khammassi and S. Krichen, "A GMDH-ANFIS approach for intrusion detection in IoT environments," *Future Generation Computer Systems*, vol. 108, pp. 1153–1164, 2020.
- [42] R. M. Cruz, R. Sabourin, and G. D. Cavalcanti, "Dynamic classifier selection: Recent advances and perspectives," *Information Fusion*, vol. 41, pp. 195–216, 2018.
- [43] C. Xu, J. Wang, and L. Zhang, "Efficient cascaded deep learning for network intrusion detection," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1581–1594, 2021.
- [44] I. Ahmad, M. Basher, M. J. Iqbal, and A. Rahim, "A hybrid ensemble approach for intrusion detection in IoT networks," *IEEE Access*, vol. 9, pp. 103 581–103 596, 2021.
- [45] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [46] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [47] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [48] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [49] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2015.
- [50] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.
- [51] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [52] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [53] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [54] D. J. Hand and R. J. Till, "A simple generalisation of the area under the ROC curve for multiple class classification problems," *Machine Learning*, vol. 45, no. 2, pp. 171–186, 2001.
- [55] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breez, M. Chukka, C. Coleman *et al.*, "MLPerf inference benchmark," in *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 446–459.
- [56] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 20, no. 2, pp. 215–232, 1958.